



GETTING STARTED GUIDE

Test Enablement Kit - Selenium

Creation date: June 30, 2025

Last modified on: July 25, 2025

Table of contents

1	Introduction	3
1.1	Folder structure	3
2	Writing test cases	4
2.1	Specify test environment	4
2.2	STEP 1: Create a new test case function / class	4
2.3	STEP 2A: Add test case logic	5
2.4	STEP 2B: Extend page object	6
3	Further topics.....	8

Glossary

Abbreviation	Description
AUT	Application Under Test
CI/CD	Continuous Integration/Continuous Delivery
OOP	Object-orientated programming
PO	Page Object
TA	Test Automation
URL	Uniform Resource Locator

1 Introduction

This „Getting started“ guide contains all the information to create the first test cases. It introduces basic concepts of Selenium tests and TestNG.

Please follow the instructions for installation in README.md to set up the environment and test folder structure.

1.1 Folder structure

This template consists of three basic folders:

- \documentation
Can be used to store diagrams and other documentation-related artifacts.
- \src
Test automation project. See below for details.
- \target
Artifacts of the Maven build process. Within this folder, the test report is generated.
- \testsuites
All TestNG test suites to test dedicated constellations.

Test cases are implemented within these folders:

Folder	Description
\src\main\	Application's source code, typically not used for test automation
\src\test\java\	Test automation source code
\src\test\resources\	Project configuration files
Package in \src\test\java\	
com.accompio.seleniumtemplate	Main package according to Maven naming convention
<...>.common	Folder with base classes
<...>.pageobjects	Folder with all page objects
<...>.reporting	Helper functions to create reports
<...>.tests	Source code of test cases

2 Writing test cases

2.1 Specify test environment

1. Open file `/src/test/resources/execution.properties`.
2. Set parameter `test.environment` to the URL of the test system.

```
test.environment=http://localhost:3000/
```

3. Set parameter `driver.type` to CHROME or FIREFOX.
When using Selenium Grid CHROME-REMOTE or FIREFOX-REMOTE.

```
driver.type=CHROME
```

4. When using Selenium Grid specify the URL and port in parameter `selenium.hub.url`.

Later, more parameters can be added to this property file, e.g., tested language.

2.2 STEP 1: Create a new test case function / class

1. **Create new class in package `com.accompio.seleniumtemplate.tests`.**

Base class must be `TestBase` to run this test with Selenium.
Test functions within this class must be marked with the annotation `@Test`.

Name and *description* should describe the idea of the test case or include references to user stories / requirements. It is used when a report is generated.

```
package com.accompio.seleniumtemplate.tests;

import com.accompio.seleniumtemplate.common.TestBase;
import org.testng.annotations.Test;

public class ExampleTest extends TestBase {
    @Test(description = "Example Test")
    protected void testExample() {
    }
}
```

Every test class may contain multiple `@Test`-functions.
Of course, it is possible to create dedicated packages for the different parts of the AUT.

2. Add the test class to test suite (/testsuites/ExampleTestSuite.xml) or create a new test suite.

The correct place is within the <classes>-section. This section may contain multiple <class> entries.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd" >
<suite name="Test Suite Transaction">
  <listeners>
    <listener class-name="com.accompio.seleniumexample.reporting.Ex-
tentTestListener"/>
  </listeners>

  <test name="Example test cases">
    <classes>
      <class name=" com.accompio.seleniumtemplate.tests.ExampleTest"/>
    </classes>
  </test>
</suite>
```

Each test class may be part of multiple test suites. Test suites may be grouped by functionality (e.g., all login-related tests) or by test layer (acceptance test, smoke test, ...). Tests are executed sequentially and may prepare preconditions for subsequent tests.

The listener is needed to create the reporting and must be added to the test suite XML file. The included listener will add screenshots to the test report in case of FAILED tests.

3. Run the tests (see README.md).

Tests are executed either via Maven or by IDE. See README.md for more details.

The test report is generated in
/target/surefire-reports/TestReport.html.

2.3 STEP 2A: Add test case logic

Now the individual test case can be implemented according to specification. This can be done either by manual interaction with browser objects or web elements (see [Write your first Selenium script | Selenium](#)) or page object pattern (which is recommended, see below).

Every test case should consist of three areas, the so-called AAA pattern:

- **Arrange:** Set up the preconditions of test case
- **Act:** Single action to test
- **Assert:** Compare actual and expected results

Please note:

- In the template browser is automatically started in function `TestBase::setup()` which is called before the test function is executed.
- It is good practice to split functional logic and page-related logic.
 - o Access to web elements should be added to [page objects](#).
 - o Assertions and test case reporting should be part of test case functionality.
- Page objects should be created when needed in test case logic. Don't instantiate them in advance. (Reason: In the constructor of `BasePO`, all page object's elements are initialized and synchronized.)

Example (with page objects):

```
InitialPagePO initialPage = new InitialPagePO();
initialPage.navigateToSecondPage();

SecondPagePO secondPage = new SecondPagePO();
secondPage.doSomething();
```

- Assertions should be taken from TestNG: [Class Assert \[JavaDoc\]](#)

Example:

```
// check success message 'Transaction submitted!'
Assert.assertTrue(aTransactionPO.isAlertTransactionSubmittedVisible(),
    "alert 'Transaction Submitted!' is not visible");
```

where `aTransactionPO` is an instance of a page object function. An example of `isAlertTransactionSubmittedVisible()` is found [below](#).

- `ReportHelper` class contains helper functions to create helpful log files, e.g., by adding tables and screenshots.

2.4 STEP 2B: Extend page object

Every *page object class* represents a web page or at least parts of it. It should be the only place within the whole test framework to interact with web pages.

Following this approach will lead to maintainable and reusable code.

In this template, page objects should be derived from the `BasePO` class.

WebElements in pageobjects

Use [@FindBy-annotation](#) to describe the needed web elements:

```
@FindBy(id = "cookie-settings-reject-all")
protected WebElement cookies_rejectAll;
```

User interactions in page objects

Every user interaction that is related to the page itself should be implemented as a separate function.

Example to enter credentials:

```
public void login(String pUsername, String pPassword) {
    inputUsername.sendKeys(pUsername);
    inputPassword.sendKeys(pPassword);
    btnSignIn.click();
}
```

Take care about synchronization after interaction. In the example above, the sign-in button is clicked and the test is immediately continued. The test case should now wait for the next expected state of UI.

To support synchronization, there are some helper functions available in the BrowserSyncUtil class.

Of course, methods can also be implemented to read state information or content of elements.

Example to check if an alert message is shown:

```
public boolean isAlertTransactionSubmittedVisible() {
    String aXPathAlert = "//div[" +
        "contains(@class, 'Alert-message') and " +
        "contains(text(), 'Transaction Submitted!')]";
    return BrowserSyncUtils.isElementVisible(aXPathAlert);
}
```

Constructor of page objects

Within constructor of the page object class, a call to `super()` should be contained. This will detect all the web elements defined in this class.

3 Further topics

After the creation of the first tests the framework can be extended according to needs.

Some additional technical notes:

- Currently the test suites are not stopped when a test case fails. When included tests are not independent, this logic can be added by adapting the TestListener classes.
- Currently the code supports Chrome and Firefox browser. It's easy to add more browsers like Edge or Safari. Please check official documentation of WebDriverManager: <https://bonigarcia.dev/webdrivermanager/>
- To run mobile tests or test Windows native applications, other frameworks can be integrated, e.g., [Appium](#).
- To add test steps for APIs, other frameworks can be integrated, e.g., [REST Assured](#).
- To get access to test data interfaces to CSV, Excel, databases or other data providers should be implemented. It is good practice to separate test logic from all needed data ("data-driven tests").
- Sensitive data like passwords should be handled with privacy and security concerns.
- To pass data from one test case to another, it is possible to use interface ITestContext functionality of TestNG.
- Integration of test automation into CI/CD is strongly recommended. Most CI/CDs can execute shell scripts (see README.md for Maven command lines).

More useful resources are:

- [Selenium: Page Object Models](#)
- [BrowserStack: Know Selenium Best Practices](#)
- [16 Selenium Best Practices For Efficient Test Automation](#)